DEPARTMENT OF AEROSPACE ENGINEERING,
SMME, NUST, ISLAMABAD

Project Report

CS – 117

Applications of ICT

Project Group Members	Class/Section	CMS ID
Ali	AE-03(B)	545895
Abdul Dayyan	AE-03(B)	557955
Asaif Imran	AE-03(B)	547828

Submitted to **Engr. Kashan Ahmad**

Table of Contents

Smart Entry Verification System for NUST

Introduction

Every day, thousands of students enter NUST through different gates. Gate 2 is one of the busiest entry points. The goal of any gate system is very simple: allow the right person to enter and stop the wrong person. But in reality, this simple task takes time, creates long lines, and still makes mistakes. Guards usually check ID cards by hand. This process depends too much on human focus, and humans get tired, distracted, or rushed. Because of this, verification becomes slow and sometimes inaccurate.

This project introduces a **Smart Entry Verification System** that uses a barcode scan and face verification together. The system was designed for NUST students entering from Gate 2. During testing, the average entry time dropped from **10 seconds to 6 seconds**, while also improving verification accuracy. The system proves that technology can make security both faster and safer.

The Problem

The traditional entry system has three major problems. First, it is slow. Each student must stop, show their card, and wait for a guard to read it. When hundreds of students arrive together, lines grow quickly. Second, it is not very accurate. A guard may miss a face, read the wrong ID, or allow someone who does not belong. Third, it depends fully on humans. Humans get tired, bored, and inconsistent, especially during peak hours.

At a university like **NUST**, where security matters a lot, these problems are serious. A faster system without accuracy is dangerous. An accurate system without speed causes frustration. We needed both speed and accuracy at the same time.

Our Solution

We built a **two-stage verification system** using a camera and software. The first stage checks the student ID using a barcode. The second stage checks the student face using face recognition. Only when *both* checks pass, access is granted.

This means even if someone steals a card, they still cannot enter unless their face matches the registered face. At the same time, scanning a barcode is very fast, so the system does not slow students down.

How the System Works

The system works in a very clear and simple flow. First, the camera looks for a barcode on the student ID card. If the barcode is valid and exists in the university database, the system accepts it. Then the system switches to face verification mode. The student is asked to look at the camera. The system captures the face and compares it with the stored face image for that CMS ID. If the face matches, access is granted. If it does not match, access is denied.

There is also a time limit of **10 seconds** for face verification. If the student does not show their face in time, the system resets. This prevents misuse and keeps the process fast.

Development Strategy

We used a **step-by-step strategy** instead of building everything at once. First, we made sure barcode scanning worked correctly. Then we tested face recognition separately. Only after both worked well alone, we combined them into one system. This reduced errors and made debugging easier.

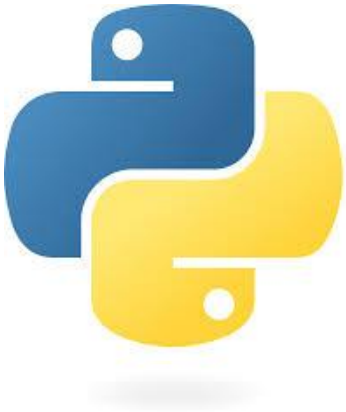
We chose this strategy because security systems must be reliable. If something breaks, we must know exactly where the problem is. A modular approach makes the system easy to understand, fix, and expand.

In addition to the step-by-step approach, we considered alternative strategies. One option was to build the full system at once, integrating barcode scanning and face recognition immediately. However, this would make it very difficult to identify errors and could slow development. Another alternative was to start with face recognition and then add barcode scanning, but we prioritized barcode scanning first because it is faster and more reliable under different lighting conditions.

We also planned for potential upgrades while developing each module. For example, the barcode module was designed so it could be replaced by QR codes or NFC tags in the future. Similarly, the face recognition module could be updated to handle multiple faces or use more advanced AI models without changing the overall system structure. This flexible, modular design ensures that our system can adapt to new technologies and future requirements.

Why We Chose This Technology

We used **Python** because it is simple, powerful, and widely used in computer vision. We used `OpenCV` for camera input and image processing. We used `pyzbar` to decode barcodes quickly and accurately. We used `face_recognition` because it provides reliable face encoding and comparison with very little code. We used **Excel files** to store CMS IDs because they are easy to manage and already used by many offices.



All these tools are open-source, which means the system is low-cost and easy to deploy in real environments like a university gate.

Project Details

Excel-Based Student Database

One key design choice was using Excel files as the student database. Excel is easy to update, widely used by administrative staff, and allows quick export/import of CMS IDs. Each CMS ID corresponds to a single student and is stored in the Excel file. During system startup, the program reads this Excel file and stores all IDs in memory as a Python list. This ensures that lookups during barcode scanning are almost instantaneous.

In-Memory Face Encodings

For face verification, we did not rely on reading the images from disk every time. Instead, we pre-processed all student images stored in a folder and generated face encodings using the `face_recognition` library. These encodings are stored in memory in a dictionary with CMS IDs as keys. When a student presents their face, the system computes the face encoding from the live camera frame and compares it immediately with the stored encoding in memory. This avoids reading images from disk repeatedly, which would have slowed down verification significantly.

Modular System Architecture

The system architecture is modular. It consists of three main modules: barcode scanning, face recognition, and state management. Barcode scanning uses `pyzbar` to decode barcodes from a grayscale frame. Face recognition uses `OpenCV` to convert BGR images to RGB and `face_recognition` to encode and compare faces. State management ensures smooth transition between barcode scanning and face verification, including handling timeouts.

Verification Timeout

We implemented a verification timeout of **10** seconds to keep the system efficient. If a student does not show their face within this window, the system resets and waits for the next barcode. This prevents delays from inattentive users and keeps the flow consistent.

Real-Time Feedback

Another technical aspect is real-time feedback. The system draws rectangles around detected barcodes and provides messages such as "BARCODE OK - SHOW FACE" or "ACCESS DENIED". During face verification, messages like "FACE NOT REGISTERED" or "ACCESS GRANTED" are displayed, providing clear guidance to users. The live camera feed continuously shows these messages using OpenCV's `putText` function.

Performance Results

Before this system, the average verification time per student was around **10 seconds**. After testing our system, the average time dropped to **6 seconds**. This may sound small, but over hundreds of students, it saves many minutes and reduces crowding.

Key observed outcomes:

- Faster student flow during peak hours.
- Reduced queue length at Gate 2.
- Lower workload on security staff.
- Higher confidence in identity verification.
- Consistent performance without human bias.

At the same time, accuracy improved because the system checks both the ID and the face. This makes impersonation very difficult.

Challenges We Faced

One major challenge was lighting. Face recognition works best with proper light. Poor lighting reduced accuracy. Another challenge was camera quality. Low-resolution cameras sometimes failed to detect faces quickly. We also faced problems with students not standing still or not looking at the camera properly. Finally, managing timeouts correctly without annoying users required careful tuning.

Additional practical challenges included:

- Inconsistent distance between the student and the camera, which affected face size and clarity.
- Motion blur when students walked too fast instead of stopping briefly.
- Barcode damage or scratches on ID cards, causing delayed scans.
- Limited processing power on normal machines, which restricted real-time performance.
- Environmental noise and crowd pressure, which made students impatient.

Possible Improvements and Expansion

This system can be improved in many ways. Multiple cameras can be added to handle more students at once. A better camera can improve face detection speed. The system can be connected to a live university database instead of Excel files. Logs can be stored to track entry times automatically. The same system can also be used for hostels, labs, and exam halls.

Further expansion ideas include:

- Adding automatic gate control so access opens without human involvement.
- Using infrared cameras for better performance at night.
- Storing multiple face images per student to improve matching accuracy.
- Adding alert logs for repeated failed attempts.
- Creating a **dashboard** for security staff to monitor live activity.
- Integrating attendance tracking with the same system.

In the future, the system can even be expanded to mobile ID verification or integration with smart gates.

Conclusion

This project shows that security does not have to be slow. By combining barcode scanning with face verification, we created a system that is faster, more accurate, and more reliable than manual checking. The reduction from **10 seconds to 6 seconds** per student proves its efficiency. More importantly, the system increases trust and safety at NUST Gate 2.

With further improvement and proper deployment, this system can become a standard security solution across the campus and beyond.

References

- `OpenCV` Documentation
- `face_recognition` Library Documentation
- `pyzbar` Library Documentation
- Python Official Documentation